UNIT 4

UNIT 4 — Structured Query Language (SQL)

1. Overview of SQL

SQL (Structured Query Language) is the standard language used to communicate with relational database management systems (RDBMS). It was originally developed at IBM (under the name SEQUEL) and has since been standardized by ANSI/ISO. All major database systems — MySQL, Oracle, MS SQL Server, PostgreSQL — support SQL, with minor variations.

SQL is a **declarative language**, which means you specify *what* data you want, not *how* to retrieve it. The DBMS engine figures out the most efficient way to execute the query.

SQL commands are grouped into five categories based on their purpose:

Category	Full Name	Purpose	Commands
DDL	Data Definition Language	Define and modify the structure of database objects	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	Insert, update, and delete data in tables	INSERT, UPDATE, DELETE
DQL	Data Query Language	Retrieve data from tables	SELECT
DCL	Data Control Language	Control user access and permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

2. Data Types in MySQL

Before creating tables, it is important to know the available data types for defining columns.

Numeric Types

Data Type	Description
`INT` / `INTEGER`	Whole numbers (positive or negative)
`BIGINT`	Very large whole numbers
`FLOAT`	Approximate decimal numbers (single precision)

Data Type	Description
`DOUBLE`	Approximate decimal numbers (double precision)
`DECIMAL(p, s)`	Exact fixed-point number with `p` total digits and `s` digits after the decimal point

String Types

Data Type	Description
`CHAR(n)`	Fixed-length string. Always stores exactly n characters. Padded with spaces if shorter.
`VARCHAR(n)`	Variable-length string. Stores up to n characters, using only the space needed.
`TEXT`	Long text data (up to 65,535 characters)
`ENUM(v1, v2, ...)`	Stores one value from a predefined list of values

Date and Time Types

Data Type	Format	Description
`DATE`	`YYYY-MM-DD`	Stores only a date
`TIME`	`HH:MM:SS`	Stores only a time
`DATETIME`	`YYYY-MM-DD HH:MM:SS`	Stores both date and time
`TIMESTAMP`	`YYYY-MM-DD HH:MM:SS`	Auto-updates to current time when a row is modified
`YEAR`	`YYYY`	Stores a year value

Boolean Type

Data Type	Description
`BOOLEAN` / `BOOL`	Stores TRUE (1) or FALSE (0)

---

3. SQL Constraints

Constraints are rules applied to columns (or tables) to enforce data integrity. They are defined when creating a table or added later using ALTER TABLE.

3.1 NOT NULL

Ensures that a column cannot store a NULL value. Every row must have a valid value for this column.

```
Name VARCHAR(50) NOT NULL
```

3.2 UNIQUE

Ensures that all values in a column are distinct — no two rows can have the same value in this column. Unlike PRIMARY KEY, a UNIQUE column can contain NULL values (usually one NULL is allowed).

```
Email VARCHAR(100) UNIQUE
```

3.3 PRIMARY KEY

Uniquely identifies each row in a table. It is essentially a combination of NOT NULL and UNIQUE. A table can have only one primary key, but the primary key can consist of multiple columns (composite key).

Column level:

```
Roll_No INT PRIMARY KEY
```

Table level:

```
PRIMARY KEY (Roll_No)
```

Composite primary key (table level only):

```
PRIMARY KEY (Student_ID, Course_ID)
```

3.4 FOREIGN KEY

Creates a link between the data in two tables. The foreign key in the child table references the primary key in the parent table. This enforces referential integrity — you cannot insert a value into the foreign key column that doesn't exist in the referenced parent table.

Column level:

```
Dept_ID INT REFERENCES Department(Dept_ID)
```

Table level:

```
FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
```

3.5 DEFAULT

Specifies a default value for a column. If no value is provided during an INSERT operation, the default value is used automatically.

```
Year INT DEFAULT 1
```

3.6 CHECK

Enforces a condition on the values that can be inserted into a column (or into the table as a whole). Every inserted or updated value must satisfy the CHECK condition.

Column level:

```
Age INT CHECK (Age >= 18)
```

Table level:

```
CHECK (Start_Date < End_Date)
```

---

4. DDL — Data Definition Language

DDL commands are used to define and modify the structure (schema) of database objects like tables, views, and indexes.

4.1 CREATE TABLE

Creates a new table in the database with specified columns, data types, and constraints.

Syntax:

```
CREATE TABLE table_name (
    column1 datatype [constraints],
    column2 datatype [constraints],
    ...
    [table_level_constraints]
);
```

Example:

```
CREATE TABLE Student (
    Roll_No   INT PRIMARY KEY,
    Name      VARCHAR(50) NOT NULL,
    Branch    VARCHAR(30),
    Year      INT DEFAULT 1,
    Email     VARCHAR(100) UNIQUE,
    Age       INT CHECK (Age >= 17)
);
```

Example with Foreign Key:

```
CREATE TABLE Enrollment (
    Roll_No    INT,
    Course_ID  VARCHAR(10),
    Grade      CHAR(2),
    PRIMARY KEY (Roll_No, Course_ID),
    FOREIGN KEY (Roll_No) REFERENCES Student(Roll_No),
```

```
    FOREIGN KEY (Course_ID) REFERENCES Course(Course_ID)
);
```

4.2 ALTER TABLE

Used to modify the structure of an existing table — adding, removing, or changing columns and constraints — without affecting the data already stored in the table.

Add a new column:

```
ALTER TABLE Student ADD Phone VARCHAR(15);
```

Drop (delete) an existing column:

```
ALTER TABLE Student DROP COLUMN Phone;
```

Modify a column's data type or size:

```
ALTER TABLE Student MODIFY COLUMN Name VARCHAR(100);
```

Rename a column:

```
ALTER TABLE Student RENAME COLUMN Name TO Full_Name;
```

Add a constraint to an existing table:

```
ALTER TABLE Student ADD CONSTRAINT chk_year CHECK (Year BETWEEN 1 AND 4);
```

Drop a named constraint:

```
ALTER TABLE Student DROP CONSTRAINT chk_year;
```

Add a foreign key after table creation:

```
ALTER TABLE Student ADD FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID);
```

4.3 DROP TABLE

Permanently and completely deletes a table from the database — both the structure (schema) and all the data stored in it are removed. This action cannot be undone.

```
DROP TABLE Student;
```

> ⚠️ **Caution:** DROP TABLE is irreversible. Once dropped, the table and all its data are permanently gone.

4.4 TRUNCATE TABLE

Removes **all rows** from a table but **keeps the table structure** (schema) intact. The table still exists, just empty. It is significantly faster than using DELETE to remove all rows because it does not log individual row deletions.

```
TRUNCATE TABLE Student;
```

> Unlike DELETE, TRUNCATE cannot be rolled back in most systems and does not fire triggers.

4.5 RENAME TABLE

Renames an existing table.

```
RENAME TABLE Student TO Learner;
```

Comparison of DROP, TRUNCATE, and DELETE

Feature	DROP	TRUNCATE	DELETE
Removes table structure?	Yes	No	No
Removes data?	Yes (all)	Yes (all rows)	Yes (specified rows)
Can be rolled back?	No	No (usually)	Yes
Can use WHERE clause?	No	No	Yes
Speed	Fast	Fast	Slower (row-by-row)

---

5. DML — Data Manipulation Language

DML commands are used to manipulate (add, change, remove) the data stored inside tables.

5.1 INSERT

Adds one or more new rows of data into a table.

Insert a row by specifying column names (recommended — order-independent):

```
INSERT INTO Student (Roll_No, Name, Branch, Year)
VALUES (101, 'Alice', 'CS', 2);
```

Insert a row for all columns in table order:

```
INSERT INTO Student
VALUES (102, 'Bob', 'IT', 3, 'bob@email.com', 20);
```

Insert multiple rows in one statement:

```sql
INSERT INTO Student (Roll_No, Name, Branch, Year) VALUES
(103, 'Carol', 'CS', 1),
(104, 'Dave',  'ECE', 4),
(105, 'Eve',   'IT', 2);
```

5.2 UPDATE

Modifies the values of one or more columns in existing rows. A WHERE clause is used to specify which rows to update.

Update a single column for a specific row:

```sql
UPDATE Student
SET Year = 3
WHERE Roll_No = 101;
```

Update multiple columns at once:

```sql
UPDATE Student
SET Branch = 'IT', Year = 2
WHERE Roll_No = 101;
```

> ⚠️ **Important:** If you omit the WHERE clause, **all rows** in the table will be updated.

5.3 DELETE

Removes one or more rows from a table. A WHERE clause specifies which rows to delete.

Delete a specific row:

```sql
DELETE FROM Student WHERE Roll_No = 104;
```

Delete rows that satisfy a condition:

```sql
DELETE FROM Student WHERE Branch = 'ECE';
```

Delete all rows (structure remains):

```sql
DELETE FROM Student;
```

> ⚠️ **Important:** DELETE without a WHERE clause removes all rows. Unlike TRUNCATE, DELETE can be rolled back within a transaction.

---

6. DQL — SELECT Statement

The SELECT statement is used to retrieve data from one or more tables. It is the most frequently used SQL command and supports a wide range of clauses for filtering, grouping, sorting, and limiting results.

6.1 Basic Syntax

```
SELECT column1, column2, ...
FROM table_name
[WHERE condition]
[GROUP BY column]
[HAVING condition]
[ORDER BY column [ASC | DESC]]
[LIMIT n];
```

6.2 Select All Columns

The `*` wildcard selects all columns from the table.

```
SELECT * FROM Student;
```

6.3 Select Specific Columns

List the column names you want to retrieve, separated by commas.

```
SELECT Name, Branch FROM Student;
```

6.4 DISTINCT — Eliminate Duplicate Rows

When a SELECT query would return duplicate values in the result, DISTINCT ensures that only unique rows are returned.

```
SELECT DISTINCT Branch FROM Student;
```

This returns each branch name only once, even if multiple students belong to the same branch.

6.5 WHERE Clause — Filter Rows

The WHERE clause filters rows based on a specified condition. Only rows that satisfy the condition are included in the result.

```
SELECT * FROM Student WHERE Branch = 'CS';

SELECT * FROM Student WHERE Year > 2 AND Branch = 'IT';

SELECT Name FROM Employee WHERE Salary >= 50000;
```

6.6 ORDER BY — Sort Results

The ORDER BY clause sorts the result set based on one or more columns. The default sort order is ascending (`ASC`). You can specify `DESC` for descending order.

```
SELECT * FROM Student ORDER BY Name ASC;
```

```sql
SELECT * FROM Employee ORDER BY Salary DESC;

-- Sort by multiple columns: first by branch, then by name within each branch
SELECT * FROM Student ORDER BY Branch ASC, Name ASC;
```

6.7 LIMIT — Restrict the Number of Rows

The LIMIT clause restricts the number of rows returned by the query.

```sql
-- Get the first 5 rows
SELECT * FROM Student LIMIT 5;

-- Get rows 6 through 10 (skip first 5, then take 5)
SELECT * FROM Student LIMIT 5 OFFSET 5;
```

---

7. SQL Operators

7.1 Arithmetic Operators

Used to perform mathematical calculations on numeric column values.

Operator	Description	Example
`+`	Addition	`Salary + 5000`
`-`	Subtraction	`Price - Discount`
`*`	Multiplication	`Quantity * Price`
`/`	Division	`Total / Count`
`%`	Modulus (remainder)	`Marks % 10`

```sql
SELECT Name, Salary * 1.10 AS Revised_Salary FROM Employee;
```

7.2 Comparison Operators

Used to compare column values. Result is TRUE or FALSE.

Operator	Meaning
`=`	Equal to
`!=` or `<>`	Not equal to
`<`	Less than
`>`	Greater than
`<=`	Less than or equal to

Operator	Meaning
`>=`	Greater than or equal to

```sql
SELECT * FROM Employee WHERE Salary > 40000;
SELECT * FROM Student WHERE Year != 1;
```

7.3 Logical Operators

Used to combine multiple conditions.

Operator	Meaning
`AND`	Both conditions must be true
`OR`	At least one condition must be true
`NOT`	Negates a condition

```sql
SELECT * FROM Student WHERE Branch = 'CS' AND Year = 2;

SELECT * FROM Student WHERE Branch = 'CS' OR Branch = 'IT';

SELECT * FROM Student WHERE NOT Year = 1;
```

7.4 Special Operators

BETWEEN ... AND

Checks whether a value falls within a specified inclusive range.

```sql
SELECT * FROM Employee WHERE Salary BETWEEN 30000 AND 60000;
-- Equivalent to: Salary >= 30000 AND Salary <= 60000
```

IN

Checks whether a value matches any value in a given list. This is cleaner than writing multiple OR conditions.

```sql
SELECT * FROM Student WHERE Branch IN ('CS', 'IT', 'ECE');
-- Equivalent to: Branch='CS' OR Branch='IT' OR Branch='ECE'
```

NOT IN

Returns rows where the value does not match any value in the list.

```sql
SELECT * FROM Student WHERE Branch NOT IN ('Civil', 'Mech');
```

LIKE — Pattern Matching

Used to search for a specified pattern in a string column. Two wildcard characters are used:

- `%` — represents zero, one, or more characters
- `_` — represents exactly one character

```sql
-- Students whose name starts with 'A'
SELECT * FROM Student WHERE Name LIKE 'A%';

-- Students whose name ends with 'son'
SELECT * FROM Student WHERE Name LIKE '%son';

-- Students whose name has 'a' as the second character
SELECT * FROM Student WHERE Name LIKE '_a%';

-- Students whose name is exactly 3 characters
SELECT * FROM Student WHERE Name LIKE '___';
```

IS NULL / IS NOT NULL

Checks whether a column value is NULL. You cannot use `= NULL` in SQL; you must use `IS NULL`.

```sql
SELECT * FROM Student WHERE Email IS NULL;

SELECT * FROM Employee WHERE Dept_ID IS NOT NULL;
```

EXISTS

The EXISTS operator checks whether a subquery returns **any rows at all**. It returns TRUE if the subquery produces at least one result, FALSE otherwise.

```sql
SELECT Name FROM Student s
WHERE EXISTS (
    SELECT 1 FROM Enrollment e WHERE e.Roll_No = s.Roll_No
);
-- Returns names of students who have at least one enrollment
```

ALL

Used with a subquery. The condition must be true for **all** values returned by the subquery.

```sql
SELECT * FROM Employee
WHERE Salary > ALL (SELECT Salary FROM Intern);
-- Returns employees whose salary is greater than every intern's salary
```

ANY

Used with a subquery. The condition must be true for **at least one** value returned by the subquery.

```sql
SELECT * FROM Employee
WHERE Salary > ANY (SELECT Salary FROM Intern);
```

```
-- Returns employees whose salary is greater than at least one intern's salary
```

8. Aggregate Functions

Aggregate functions perform a calculation on a **set of values** and return a single summary value. They are typically used with GROUP BY.

Function	Description
`COUNT(*)`	Counts the total number of rows in the result
`COUNT(column)`	Counts the number of non-NULL values in a column
`SUM(column)`	Returns the sum of all values in a numeric column
`AVG(column)`	Returns the average of all non-NULL values in a numeric column
`MAX(column)`	Returns the maximum value in a column
`MIN(column)`	Returns the minimum value in a column

```sql
-- Total number of students
SELECT COUNT(*) FROM Student;

-- Number of students who have an email address on record
SELECT COUNT(Email) FROM Student;

-- Average salary of all employees
SELECT AVG(Salary) FROM Employee;

-- Highest and lowest salary
SELECT MAX(Salary), MIN(Salary) FROM Employee;

-- Total marks scored by student with Roll_No 101
SELECT SUM(Marks) FROM Result WHERE Roll_No = 101;
```

9. GROUP BY and HAVING

9.1 GROUP BY

The GROUP BY clause groups the rows of a table that have the **same values in specified columns** into summary rows. It is almost always used with aggregate functions to produce a summary per group.

```sql
-- Count of students in each branch
SELECT Branch, COUNT(*) AS Total_Students
FROM Student
GROUP BY Branch;
```

```sql
-- Average salary in each department
SELECT Dept_ID, AVG(Salary) AS Avg_Salary
FROM Employee
GROUP BY Dept_ID;
```

> **Rule:** Any column that appears in the SELECT clause but is not inside an aggregate function **must** appear in the GROUP BY clause.

9.2 HAVING

The HAVING clause is used to **filter groups** produced by GROUP BY based on aggregate conditions. It is analogous to the WHERE clause but operates on groups rather than individual rows.

```sql
-- Only show branches with more than 20 students
SELECT Branch, COUNT(*) AS Total
FROM Student
GROUP BY Branch
HAVING COUNT(*) > 20;
```

```sql
-- Departments where average salary exceeds 50000
SELECT Dept_ID, AVG(Salary) AS Avg_Sal
FROM Employee
GROUP BY Dept_ID
HAVING AVG(Salary) > 50000;
```

> **Key distinction:**
>
> - **WHERE** filters individual rows **before** grouping
> - **HAVING** filters groups **after** grouping

---

10. Joins

Joins are used to combine rows from two or more tables based on a related column. They allow you to retrieve data that is spread across multiple tables in a single query.

10.1 CROSS JOIN (Cartesian Product)

A CROSS JOIN returns the Cartesian product of two tables — every row from the first table is combined with every row from the second table. No join condition is specified.

```sql
SELECT * FROM Student CROSS JOIN Course;
```

If Student has 50 rows and Course has 10 rows, the result has 500 rows. In practice, CROSS JOIN is rarely used directly; it is mostly useful as a basis for understanding other joins.

10.2 INNER JOIN

An INNER JOIN returns only those rows where the join condition is satisfied in **both** tables. Rows that do not have a matching entry in the other table are excluded from the result.

```
SELECT Student.Name, Department.Dept_Name
FROM Student
INNER JOIN Department ON Student.Dept_ID = Department.Dept_ID;

SELECT s.Name, d.Dept_Name
FROM Student AS s
INNER JOIN Department AS d ON s.Dept_ID = d.Dept_ID;
```

This returns only students who have been assigned to a department that exists in the Department table.

Equi-Join

A specific type of inner join where the condition uses the equality operator (`=`). This is the most common form of join.

```
SELECT e.Name, d.Dept_Name
FROM Employee e
INNER JOIN Department d ON e.Dept_ID = d.Dept_ID;
```

Non-Equi Join

A join where the condition uses an operator other than `=`, such as `<`, `>`, `BETWEEN`, etc.

```
SELECT e.Name, g.Grade_Label
FROM Employee e
INNER JOIN Grade g ON e.Salary BETWEEN g.Min_Sal AND g.Max_Sal;
```

10.3 NATURAL JOIN

A NATURAL JOIN automatically joins two tables based on **all columns that have the same name** in both tables. The duplicate common columns are merged into a single column in the result.

```
SELECT * FROM Student NATURAL JOIN Department;
```

If both tables have a column named `Dept_ID`, the join is performed on that column and only one `Dept_ID` column appears in the result.

10.4 LEFT OUTER JOIN (LEFT JOIN)

Returns **all rows from the left table** and the matched rows from the right table. Where there is no match in the right table, NULL values are filled in for all right-table columns.

```
SELECT Student.Name, Department.Dept_Name
FROM Student
```

```
LEFT JOIN Department ON Student.Dept_ID = Department.Dept_ID;
```

Students who have not been assigned to any department will still appear in the result, with NULL in the `Dept_Name` column.

10.5 RIGHT OUTER JOIN (RIGHT JOIN)

Returns **all rows from the right table** and the matched rows from the left table. Where there is no match in the left table, NULL values are filled in for all left-table columns.

```
SELECT Student.Name, Department.Dept_Name
FROM Student
RIGHT JOIN Department ON Student.Dept_ID = Department.Dept_ID;
```

Departments that have no students assigned will still appear, with NULL in the `Name` column.

10.6 FULL OUTER JOIN

Returns **all rows from both tables**. Rows with no match in the other table get NULL values for the unmatched columns on the opposite side.

```
SELECT Student.Name, Department.Dept_Name
FROM Student
FULL OUTER JOIN Department ON Student.Dept_ID = Department.Dept_ID;
```

> **Note:** MySQL does not directly support FULL OUTER JOIN. It is simulated by doing a LEFT JOIN combined with a UNION of a RIGHT JOIN:
>
> ```
> SELECT * FROM Student LEFT JOIN Department ON Student.Dept_ID =
> Department.Dept_ID
> UNION
> SELECT * FROM Student RIGHT JOIN Department ON Student.Dept_ID =
> Department.Dept_ID;
> ```

10.7 SELF JOIN

A SELF JOIN is when a table is joined with **itself**. This is useful when a table contains hierarchical or recursive data, such as an employee-manager relationship where both employees and their managers are stored in the same table.

```
SELECT e1.Name AS Employee, e2.Name AS Manager
FROM Employee e1
INNER JOIN Employee e2 ON e1.Manager_ID = e2.Emp_ID;
```

Here, the Employee table is referred to by two different aliases (`e1` for the employee, `e2` for the manager) to distinguish the two "copies" being joined.

Join Summary

Join Type	What it Returns
CROSS JOIN	All possible row combinations from both tables
INNER JOIN	Only rows with matching values in both tables
LEFT JOIN	All rows from left table + matched rows from right (NULLs for unmatched right)
RIGHT JOIN	All rows from right table + matched rows from left (NULLs for unmatched left)
FULL OUTER JOIN	All rows from both tables (NULLs where no match on either side)
SELF JOIN	Table joined with itself (for hierarchical data)

---

11. Subqueries (Nested Queries)

A subquery is a SELECT statement that is **embedded inside another SQL statement** (the outer query). The inner query (subquery) executes first, and its result is used as input to the outer query.

Subqueries can appear in the WHERE, HAVING, or FROM clause of the outer query.

Single-Row Subquery

Returns exactly one row and one column. Used with single-value comparison operators.

```sql
-- Find the employee with the highest salary
SELECT Name FROM Employee
WHERE Salary = (SELECT MAX(Salary) FROM Employee);
```

Multi-Row Subquery

Returns multiple rows. Must be used with operators like IN, ALL, or ANY.

```sql
-- Find students enrolled in course 'CS101'
SELECT Name FROM Student
WHERE Roll_No IN (
    SELECT Roll_No FROM Enrollment WHERE Course_ID = 'CS101'
);
```

Correlated Subquery

A correlated subquery references a column from the outer query. Unlike a regular subquery that runs once, a correlated subquery runs **once for each row** processed by the outer query.

```sql
-- Find employees who earn more than the average salary in their own department
SELECT Name FROM Employee e1
WHERE Salary > (
    SELECT AVG(Salary) FROM Employee e2
    WHERE e1.Dept_ID = e2.Dept_ID
);
```

---

12. Set Operations

Set operations combine the results of two or more SELECT statements. Both queries must be **union-compatible** — same number of columns and compatible data types.

UNION

Returns all distinct rows from both queries. Duplicate rows are automatically removed.

```sql
SELECT Name FROM UG_Student
UNION
SELECT Name FROM PG_Student;
```

UNION ALL

Returns all rows from both queries, **including duplicates**. Faster than UNION since it skips duplicate elimination.

```sql
SELECT Name FROM UG_Student
UNION ALL
SELECT Name FROM PG_Student;
```

INTERSECT

Returns only rows that appear in **both** query results.

```sql
SELECT Roll_No FROM Enrolled_CS
INTERSECT
SELECT Roll_No FROM Enrolled_IT;
```

EXCEPT (or MINUS in Oracle)

Returns rows from the first query result that do **not** appear in the second.

```sql
SELECT Name FROM All_Employees
EXCEPT
SELECT Name FROM Resigned_Employees;
```

---

13. Views

A **view** is a virtual table defined by a stored SELECT query. The view does not store any data of its own — it simply stores the query definition. When the view is queried, the underlying SELECT statement is executed and the result is returned.

Views are used to:

- **Simplify complex queries** by giving them a simple name
- **Enforce security** by exposing only specific columns or rows to certain users
- **Provide a consistent interface** — even if the underlying table structure changes, the view can be updated to hide the change

Create a View

```
CREATE VIEW CS_Students AS
SELECT Roll_No, Name, Year
FROM Student
WHERE Branch = 'CS';
```

Query a View

Once created, a view can be used exactly like a table in a SELECT statement.

```
SELECT * FROM CS_Students;

SELECT Name FROM CS_Students WHERE Year = 2;
```

Replace / Update a View

```
CREATE OR REPLACE VIEW CS_Students AS
SELECT Roll_No, Name, Year, Email
FROM Student
WHERE Branch = 'CS';
```

Drop a View

```
DROP VIEW CS_Students;
```

> Dropping a view does not affect the underlying table or its data.

---

14. SQL Aliases

Aliases provide a **temporary, alternative name** for a table or column within a query. They are used to make queries more readable and to shorten long table names, especially when doing self-joins or working with long expressions.

Column Alias

Renames a column in the output result.

```
SELECT Name AS Student_Name, Salary * 1.1 AS Revised_Salary
FROM Employee;
```

Table Alias

Gives a table a short nickname for use within the query.

```
SELECT e.Name, d.Dept_Name
FROM Employee AS e
INNER JOIN Department AS d ON e.Dept_ID = d.Dept_ID;
```

> The keyword `AS` is optional — `Employee e` and `Employee AS e` are equivalent.

---

15. DCL — Data Control Language

DCL commands are used by database administrators to control access to the database — granting or revoking permissions for users.

GRANT

Gives a specific user permission to perform certain operations on database objects.

```
-- Give user 'john' permission to read and add data to the Student table
GRANT SELECT, INSERT ON Student TO 'john'@'localhost';

-- Give user 'admin' all privileges on the entire database
GRANT ALL PRIVILEGES ON mydb.* TO 'admin'@'localhost';
```

REVOKE

Removes previously granted permissions from a user.

```
-- Remove insert permission from user 'john' on the Student table
REVOKE INSERT ON Student FROM 'john'@'localhost';
```

---

16. TCL — Transaction Control Language

A **transaction** is a sequence of one or more SQL operations that are treated as a **single logical unit of work**. Either all operations in the transaction succeed (and are permanently saved), or none of them take effect (and the database is returned to its prior state).

TCL commands control the lifecycle of transactions.

COMMIT

Permanently saves all changes made during the current transaction to the database. Once committed, the changes cannot be undone.

```
COMMIT;
```

ROLLBACK

Undoes all changes made since the last COMMIT. The database is restored to the state it was in at the start of the transaction (or at the last SAVEPOINT).

```
ROLLBACK;
```

SAVEPOINT

Creates a named checkpoint (marker) within a transaction. You can later roll back to a specific SAVEPOINT without undoing the entire transaction.

```
SAVEPOINT sp1;
```

Rolling back to a SAVEPOINT:

```
ROLLBACK TO sp1;
-- This undoes only the operations performed after SAVEPOINT sp1
-- Operations before sp1 are preserved
```

Example: Using All TCL Commands Together

```
START TRANSACTION;

INSERT INTO Account (Acc_ID, Balance) VALUES (201, 10000);

SAVEPOINT after_insert;

UPDATE Account SET Balance = Balance - 2000 WHERE Acc_ID = 201;

-- Something went wrong with the update — undo just that operation
ROLLBACK TO after_insert;

-- The INSERT is still in effect; commit it permanently
COMMIT;
```

---

17. Execution Order of a SELECT Query

When you write a SELECT statement, the clauses appear in this written order:

```
SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
```

However, the database engine **executes** them in a different order:

```
FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
```

Understanding this execution order is important because it explains, for example, why you cannot use a column alias (defined in SELECT) inside a WHERE clause — the WHERE clause is evaluated before SELECT assigns the alias.
